

# VOID: execution plan, in workflow sized pieces

Companion to `BUILD-PLAN.md` . That document says what to build and in which order. This one says how each piece gets built by Claude running subagents, so that any package can be started with one sentence and finished with evidence.

Written 4 September 2026.

## 1. How to read this

The build is cut into **work packages** (WP). Each one is sized for one or two runs of the Workflow tool, which fans work out across subagents with deterministic control flow. The sizing rule applied throughout:

- **One workflow is one well scoped fan out:** at most 15 agents, one clear question, one merge at the end.
- **Multi phase work is several workflows in sequence,** with a human readable result between them, so the person driving stays in the loop and can redirect before the next fan out spends anything.
- **Nothing merges without an adversarial pass.** Every package ends with agents whose only job is to break what was built. This is not optional because the product is a security layer: a plausible but wrong interceptor is worse than none.
- **Every exit criterion is a command that runs.** If it cannot be run, the package is not done.

Each package lists:

| FIELD    | MEANING                                                              |
|----------|----------------------------------------------------------------------|
| Goal     | The one sentence the package exists for                              |
| Needs    | Packages that must be complete first, and anything required from you |
| Shape    | Phases, agent counts and the orchestration pattern per phase         |
| Agents   | What each agent is told to do, in one line                           |
| Produces | Files and artifacts that exist afterwards                            |
| Exit     | Commands that must pass                                              |
| Budget   | Rough token spend for the whole package                              |
| Traps    | What has gone wrong before in work like this                         |

Token figures are for planning. They assume roughly 60k to 150k tokens per agent depending on how much it reads, and they are deliberately rounded up.

## The protocol per package

1. You say **"run WP-NN"**. That sentence is the opt in for multi agent orchestration, which the tooling requires explicitly each time.
2. I scout inline first: read what exists, list the files, confirm the prerequisites, then write the workflow script.
3. The workflow runs. Progress is visible as it goes.
4. I run every exit criterion myself, in this environment, and show the output.
5. I commit on the feature branch with the evidence in the message, and push.
6. You read the exit evidence and say go, or redirect. Nothing in the next package starts until then.

## The context pack

Every agent in every workflow reads the same short set of files before doing anything. This is the stable prefix from `BUILD-PLAN.md` section 3, made concrete, and WP-00 creates it:

|                                   |                                                   |
|-----------------------------------|---------------------------------------------------|
| <code>docs/CONTEXT.md</code>      | what VOID is, in 400 words, plus the vocabulary   |
| <code>docs/DECISIONS.md</code>    | the decision log, one paragraph per decision      |
| <code>docs/STANDARDS.md</code>    | coding rules, test rules, commit rules, no dashes |
| <code>packages/*/README.md</code> | the contract of each package, kept to one screen  |

Agents are told the pack is authoritative and that the code must follow it, not the other way round. When the pack is wrong, the fix is a package that updates the pack, never an agent that quietly diverges.

## 2. Ground rules applied in every workflow

**Worktree isolation for parallel writers.** Any phase where more than one agent edits files runs each agent in its own git worktree. Merging is a separate, single agent step that resolves conflicts and runs the typecheck. Without this, two agents editing `packages/proxy/src/index.ts` corrupt each other silently.

**Structured output everywhere a result feeds another stage.** Reviewers return `{ findings: [{ file, line, severity, claim, repro }] }`, not prose. Judges return scores. Verifiers return `{ refuted: boolean, reason }`. This is what lets the script decide without a model reading paragraphs.

**Adversarial verification before merge.** Three independent skeptics per significant claim, each prompted to refute it and to default to refuted when unsure. A claim survives with two of three.

For security relevant packages (proxy, ledger, policy, probes) the skeptics get distinct lenses rather than the same prompt three times: correctness, security, and does it actually reproduce.

**Real systems in verification, mocks only in unit tests.** The verification agent for a connector runs a real Postgres in this environment, makes a real change through the proxy, replays, and compares rows with `psql`. A green test against a mocked client proves nothing about an undo.

**Pipeline by default.** Items flow through stages independently. A barrier appears only where a stage genuinely needs every result of the previous one, which in this plan is: dedup before verification, and the merge step after parallel writers.

**One package, one branch, one commit series.** Each package lands as a small number of commits on `claude/poradna-design-animations-wwbc11` and is pushed to `main` only after its exit criteria have been shown. Never the other way.

**Nothing autonomous touches a production account.** Test databases run locally. Cloud verification uses a disposable account with a spend cap that you create and hand over. This rule is in `STANDARDS.md` and every agent reads it.

### 3. Dependency graph

```
WP-00 context pack and scaffold
|
+-- WP-01 proxy, stdio, core messages
|   |
|   +-- WP-02 proxy, streamable HTTP, progress, cancel, reconnect
|
+-- WP-03 ledger
|
+-- WP-04a registry runtime and evaluator
|
+-- WP-04b registry expansion by batch (any time after 04a)
|
+-- WP-05 policy and blocking hold      (needs 01, 03, 04a)
|
+-- WP-06 connector: Postgres          (needs 05)
+-- WP-07 connector: S3                (needs 05)
|
+-- WP-08 probes and blast radius      (needs 06, 07)
+-- WP-09 CLI                          (needs 06)
|
+-- WP-10 control plane API            (needs 09)
|
+-- WP-11 control plane UI
+-- WP-12 taint graph (two workflows)
+-- WP-13 attestation and standalone verifier
|
+-- WP-14 SDK wrap
+-- WP-15 hardening, docs, release
```

WP-01 and WP-03 and WP-04a are independent and can run in the same session, one after another, without waiting on each other's review. Everything from WP-05 on is strictly sequential because each one is verified against the real behaviour of the one before it.

### 4. The packages

WP-00: context pack and scaffold

**Goal.** Turn the marketing repository into a workspace the product can grow in, and write the four files every later agent will read first.

**Needs.** Nothing. This is the first thing that runs.

**Shape.** One workflow, three phases, 9 agents.

| PHASE      | PATTERN                                               | AGENTS         |
|------------|-------------------------------------------------------|----------------|
| Understand | 4 parallel readers, one per subsystem, structured map | 4              |
| Decide     | judge panel: 3 independent decision logs, 1 synthesis | 4              |
| Scaffold   | 1 builder in a worktree, then 1 verifier              | 2 (sequential) |

**Agents.**

- Reader A: `api/` and `artifacts/api-server/` . Return every function worth lifting into a package, with file and line: `sealHash` , the rate limiter, the store bootstrap, the static site mount.
- Reader B: `api/_registry.ts` and `_registry_http.ts` . Return the type contract and every consumer of it.
- Reader C: `lib/db/` and the Drizzle schema. Return what exists, what the ledger table will need to add.
- Reader D: `artifacts/void/` . Return the design tokens and the component primitives the control plane UI will reuse, so WP-11 does not reinvent them.
- Deciders 1 to 3: each writes a full decision log for the five decisions in `BUILD-PLAN.md` section 5 phase 0, from a different bias: fastest to demo, safest to operate, cheapest to run. Each returns the log plus a one line reason per decision.
- Synthesiser: reads the three logs and the readers' maps, writes `docs/DECISIONS.md` choosing per decision and recording the losing options and why, then writes `docs/CONTEXT.md` and `docs/STANDARDS.md` .
- Builder: creates `packages/{registry,ledger,proxy,policy,connectors,cli}` with `package.json` , `tsconfig.json` , `README.md` (contract only), one trivial test each. Moves `api/_registry.ts` to `packages/registry/src` and makes `api/_registry.ts` a re-export so the Vercel Function and the site keep working. Adds the workspace test runner.
- Verifier: runs typecheck, `pnpm test` , `pnpm run build:web` , then opens `/registry` in a browser and confirms 89 rows and the same stats as before.

**Produces.** `packages/*` skeleton, `docs/CONTEXT.md` , `docs/DECISIONS.md` , `docs/STANDARDS.md` , a passing test command.

**Exit.**

```
pnpm run typecheck
pnpm test                # one trivial test per package, all green
pnpm run build:web       # site still builds
curl localhost:8080/api/registry?view=stats | grep '"entries":89'
```

**Budget.** About 700k tokens.

**Traps.** The re-export from `api/_registry.ts` must stay dependency free or the Vercel Function stops bundling. The verifier checks this by running the real emit: `tsc -p api/tsconfig.json --outDir /tmp/emit`.

WP-01: MCP proxy, stdio transport, core messages

**Goal.** A proxy that terminates the agent's MCP connection as a server, opens its own connection to the upstream server as a client, and forwards `initialize`, `tools/*`, `resources/*`, `prompts/*` and their notifications without changing anything the agent can observe.

**Needs.** WP-00. Also the `@modelcontextprotocol/sdk` package installed; the scout step confirms the version and pins it.

**Shape.** One workflow, four phases, 14 agents.

| PHASE     | PATTERN                                                            | AGENTS |
|-----------|--------------------------------------------------------------------|--------|
| Spec      | 2 readers of the MCP specification, structured message inventory   | 2      |
| Design    | 3 independent interface designs, 1 judge and synthesis             | 4      |
| Implement | pipeline over 4 modules, each builder in a worktree, then 1 merger | 5      |
| Verify    | 1 integration runner, 2 lens reviewers                             | 3      |

**Agents.**

- Spec reader A: base protocol, lifecycle, capability negotiation, error codes. Returns every request and notification type with its direction.
- Spec reader B: tools, resources, prompts, list changed notifications, pagination cursors. Same shape.
- Designers 1 to 3: given both inventories, design the module boundary and the `Session` interface from three biases: minimal surface, maximum observability, easiest to add a second transport. Return interfaces as code.
- Judge: scores the three on the four criteria in `DECISIONS.md`, picks one, grafts the best idea from each runner up, writes `packages/proxy/README.md` with the frozen interface.
- Builders, one per module, each in its own worktree: `transport/stdio.ts` (framing, partial reads, backpressure), `session.ts` (handshake, capability forwarding, id remapping), `forward/tools.ts` and `forward/resources.ts`, `relay/notifications.ts` (both directions, list changed). Each writes unit tests against the frozen interface.
- Merger: pulls the four worktrees together, resolves conflicts, runs typecheck and tests, and does not add features.
- Integration runner: installs a real MCP server (the reference filesystem server is enough), runs a scripted 30 call session direct and through the proxy, records both transcripts, returns the diff. Any difference other than timing fails the package.

- Reviewer, protocol lens: reads the diff and the code, tries to find a message that would be forwarded wrongly. Returns findings with repro.
- Reviewer, failure lens: kills the upstream mid call, sends malformed JSON, sends a request before initialize. Returns what the proxy did and whether the agent saw a clean error.

**Produces.** `packages/proxy` with stdio transport, session, forwarders, notification relay, tests, and a `void-proxy` binary that takes an upstream command.

**Exit.**

```
pnpm --filter @void/proxy test
node scripts/transcript-diff.mjs --direct --via-proxy # prints "0 differences"
```

**Budget.** About 1.5M tokens.

**Traps.** Request id remapping. The agent and the upstream each number their own requests; a proxy that forwards ids unchanged will eventually collide on server initiated requests. The `session.ts` builder is told this explicitly.

WP-02: MCP proxy, Streamable HTTP, progress, cancellation, reconnect

**Goal.** The same proxy over HTTP, plus the three behaviours that produce hung agents when they are missing.

**Needs.** WP-01.

**Shape.** One workflow, three phases, 11 agents.

| PHASE     | PATTERN                                        | AGENTS |
|-----------|------------------------------------------------|--------|
| Implement | pipeline over 4 modules in worktrees, 1 merger | 5      |
| Verify    | 3 scenario runners, one per behaviour          | 3      |
| Break     | 3 adversarial reviewers with distinct lenses   | 3      |

**Agents.**

- Builders: `transport/http.ts` (Streamable HTTP, session header, resumable streams), `progress.ts` (progress token passthrough, streamed partial results), `cancel.ts` (cancellation forwarded upstream and the pending request cleaned up), `reconnect.ts` (upstream drops, proxy reconnects, in flight requests fail cleanly, agent session survives).
- Scenario runners: one each for a long call with progress, a cancelled call, and an upstream restart during a call. Each returns the transcript and a pass or fail against the behaviour written in the proxy README.
- Reviewers: concurrency lens (two calls in flight, one cancelled), resource lens (does a dropped connection leak the upstream process), and protocol lens (session id handling on reconnect).

**Produces.** Both transports, and `void-proxy --http :7777`.

**Exit.**

```
pnpm --filter @void/proxy test
node scripts/scenario.mjs progress      # streams, completes
node scripts/scenario.mjs cancel        # upstream saw the cancel within 100ms
node scripts/scenario.mjs reconnect     # in flight call errored, next call succeeded
```

**Budget.** About 1.2M tokens.

**Traps.** Cancellation is where most proxies fail silently: the agent gives up, the proxy keeps the upstream call alive, and the next call queues behind it. The scenario runner measures the time from cancel to upstream receipt.

WP-03: the ledger

**Goal.** An append only, hash chained, signed record with a verifier that catches both an edited body and a removed entry, because those are different failures.

**Needs.** WP-00. Independent of the proxy.

**Shape.** One workflow, three phases, 9 agents.

| PHASE     | PATTERN                                        | AGENTS |
|-----------|------------------------------------------------|--------|
| Implement | pipeline over 3 modules in worktrees, 1 merger | 4      |
| Verify    | 1 runner for the three commands                | 1      |
| Break     | 4 forgers, each trying a different attack      | 4      |

**Agents.**

- Builders: `chain.ts` (lift `sealHash` from `api/_core.ts`, canonical form, link), `store.ts` (Postgres append with a `BEFORE UPDATE OR DELETE` trigger that raises, plus a local JSONL store for the dev tier), `sign.ts` (ed25519 over the head, key from env, `KeyProvider` interface with a KMS stub).
- Runner: appends 1000 entries through the public API and runs the three verify commands.
- Forgers: edit a body and leave the hash, delete an entry, reorder two entries, and append an entry with a forged signature. Each returns whether the verifier caught it. Any uncaught forgery fails the package.

**Produces.** `packages/ledger`, `void ledger verify`.

**Exit.**



```
void ledger verify          # 1000 entries, chain intact, signature valid
void ledger verify --tamper 42 # body check fails at 42
void ledger verify --drop 42  # link check fails at 43
psql -c "UPDATE ledger SET klass='r0' WHERE seq=42" # trigger raises
```

**Budget.** About 900k tokens.

**Traps.** Canonical JSON. If key order changes the hash, two runtimes will disagree about the same entry. The chain builder sorts keys and the forger that reorders fields is there to prove it.

## WP-04a: registry runtime and evaluator

**Goal.** The registry stops being a lookup table and becomes an evaluator: tool id plus arguments plus a context of declared facts in, the first matching case out.

**Needs.** WP-00.

**Shape.** One workflow, three phases, 10 agents.

| PHASE     | PATTERN                                                     | AGENTS |
|-----------|-------------------------------------------------------------|--------|
| Design    | 2 independent designs of the precondition language, 1 judge | 3      |
| Implement | pipeline over 3 modules in worktrees, 1 merger              | 4      |
| Break     | 3 skeptics feeding adversarial inputs                       | 3      |

### Agents.

- Designers: the `when` strings in the registry are prose today. Each designer proposes a small structured form that can be evaluated ( `{ fact: "versioning", is: "enabled" }` ) while keeping the prose for display, and shows how all 178 existing cases translate. One is biased to the smallest grammar, one to the most expressive.
- Judge: picks, writes the JSON schema into `packages/registry/schema.json`.
- Builders: `evaluate.ts` (first case whose facts hold, with the unguarded fallback), `facts.ts` (declared facts config loader with `verified_at` and staleness warnings), `migrate.ts` (one shot translation of the 178 prose cases into structured form, output reviewed by a human before merge).
- Skeptics: feed missing facts, contradictory facts, unknown tool ids, and arguments that should change the class (a `versionId` on an S3 delete). Each returns cases where the evaluator answered wrongly.

**Produces.** `packages/registry` with `evaluate()`, `schema.json`, the migrated registry, and a `void classify --dry-run` command that prints the class distribution over a transcript.

**Exit.**

```
pnpm --filter @void/registry test
void classify --transcript fixtures/session.jsonl
# prints a distribution; unclassified below 20 percent on the fixture
```

**Budget.** About 1.1M tokens.

**Traps.** The migration of prose to structure is the one place a human must read the output line by line. The workflow stops after `migrate.ts` writes its proposal and does not merge until you have skimmed it.

## WP-04b: registry expansion by batch

**Goal.** Grow the registry from 89 to roughly 250 entries with the same quality bar, using batch generation with structured outputs and adversarial review per entry.

**Needs.** WP-04a. Can run any time after, including overnight.

**Shape.** One workflow, three phases. Agent count is per entry, so this is the one package that exceeds the 15 agent guideline: about 160 entries times three agents is roughly 500 agents, run 10 at a time. You opt in to that size explicitly when you start it, or split it by vendor group into four runs.

| PHASE               | PATTERN                                                          | AGENTS      |
|---------------------|------------------------------------------------------------------|-------------|
| Inventory           | 5 parallel listers, one per vendor group, dedup barrier          | 5           |
| Generate and verify | pipeline per candidate: 1 writer, 2 refuters against vendor docs | 3 per entry |
| Merge               | 1 agent writes the entries, 1 verifier runs schema and stats     | 2           |

## Agents.

- Listers: cloud, payments and commerce, messaging and productivity, data and infrastructure, identity and AI. Each returns candidate call ids with a one line summary, excluding anything already in the registry.
- Writer per candidate: writes the entry against `schema.json`, ordered cases, unguarded fallback last, with the precondition in structured form.
- Refuters per candidate: fetch the vendor documentation and try to show a case is wrong, a window is wrong, or a precondition is not checkable. Entry survives only if both fail to refute.
- Merge writer: appends survivors, sorted, to the registry source.
- Verifier: schema validation, duplicate id check, stats before and after.

**Produces.** Registry version `2026.09.2` with roughly 250 entries and a `docs/registry-changelog.md` listing what was added and what was refuted.

## Exit.

```
pnpm --filter @void/registry test
node -e "... " # prints entries >= 240, dupes 0, every entry ends with an unguarded case
```

**Budget.** About 8M tokens if run as one job. This is the package where the batch tier in `BUILD-PLAN.md` section 3 matters.

**Traps.** Refuters need real vendor pages. When a page is behind a login or returns 403, the entry is marked unverified and excluded, and the log says so. Silent inclusion of an unverified entry is the failure mode.

## WP-05: policy engine and the blocking hold

**Goal.** Rules in, a decision out, and the third decision holds the call until a human acts or the timer runs out. This is the package that produces the moment described in `BUILD-PLAN.md` phase 4.

**Needs.** WP-01, WP-03, WP-04a. From you: nothing yet. Slack approval is built against a webhook you can provide later; the CLI approval channel is enough for the exit criterion.

**Shape.** One workflow, four phases, 14 agents.

| PHASE     | PATTERN                                        | AGENTS |
|-----------|------------------------------------------------|--------|
| Design    | 2 rule format proposals, 1 judge               | 3      |
| Implement | pipeline over 5 modules in worktrees, 1 merger | 6      |
| Verify    | 1 end to end runner for the moment             | 1      |
| Break     | 4 skeptics                                     | 4      |

## Agents.

- Designers: YAML match rules as in `BUILD-PLAN.md` phase 4, one biased to the smallest useful set of match keys, one to expressiveness. Judge picks and freezes `packages/policy/README.md`.
- Builders: `rules.ts` (load, validate, first match wins), `decide.ts` (class and blast radius and tool in, allow or deny or hold out), `hold.ts` (pending map, timer, resolve or expire, the MCP error body on cancel that a model can read), `channels/cli.ts` (a `void approvals` command that lists and resolves), `channels/slack.ts` (webhook post with two buttons, behind a feature flag until tested).
- End to end runner: starts a local Postgres and a Postgres MCP server, runs an agent script that attempts `DELETE FROM orders`, observes the hold in the CLI, cancels it, confirms the table is unchanged, and returns the agent's next message verbatim so a human can judge whether it recovered sensibly.
- Skeptics: a rule that never matches, a hold that expires during a proxy restart, two holds resolving out of order, and an approval from an unauthenticated caller.

**Produces.** `packages/policy`, `void approvals`, the first policy file `policy/default.yaml`.

**Exit.**

```
pnpm --filter @void/policy test
node scripts/moment.mjs
# expected output:
#   held: postgres.row.delete (r1, 41883 rows)  waiting for approval
#   cancelled by cli
#   orders unchanged: 41883 rows
#   agent said: "The delete was blocked by policy: ..."
```

**Budget.** About 1.6M tokens.

**Traps.** The MCP error on cancel. If the proxy returns a generic error, the agent retries, sometimes with a slightly different statement, and you have built a tool that trains agents to evade it. The error body names the rule, the class, and what would be needed for approval, and the runner returns the agent's reaction so you can read it.

WP-06: connector, Postgres

**Goal.** The first real undo. Before images for `UPDATE` , full rows plus cascade for `DELETE` , replay in dependency order, and a snapshot store that keeps customer data in the customer's bucket.

**Needs.** WP-05. A local Postgres, which this environment has.

**Shape.** One workflow, four phases, 13 agents.

| PHASE     | PATTERN                                                                   | AGENTS |
|-----------|---------------------------------------------------------------------------|--------|
| Design    | 1 designer for the connector interface, 1 for the snapshot store, 1 judge | 3      |
| Implement | pipeline over 4 modules in worktrees, 1 merger                            | 5      |
| Verify    | 1 runner that uses psql, not assertions                                   | 1      |
| Break     | 4 skeptics                                                                | 4      |

**Agents.**

- Designers: the `Connector` interface ( `classify` , `capture` , `inverse` , `apply` ) and the `SnapshotStore` interface (put with digest, get by reference, retention, redaction hook). Judge freezes both READMEs. These two interfaces are the ones every later connector implements, so they get the most design attention in the plan.
- Builders: `postgres/parse.ts` (statement type, target tables, predicate extraction; uses a real SQL parser, not regex), `postgres/capture.ts` (keyed `SELECT` before image for `UPDATE` , full rows for `DELETE` , `information_schema` walk for `ON DELETE CASCADE` ), `postgres/inverse.ts` (keyed `UPDATE` from image, ordered `INSERT` with identity columns preserved),

`snapshot/local.ts` and `snapshot/s3.ts` (the store, with redaction applied before the bytes leave the process).

- Runner: seeds `orders`, `order_items`, `refund_queue` with the cascade from the site's blast radius demo, runs the agent script through the proxy to delete draft orders, runs `void replay --to <before>`, then queries all three tables with `psql` and diffs against a dump taken before. Returns the diff. Empty diff passes.
- Skeptics: a `DELETE` with a subquery predicate, a table with a composite primary key, a row that was changed by a human between capture and replay, and a snapshot bucket that rejects the write.

**Produces.** `packages/connectors/postgres`, `packages/connectors/snapshot`, `void replay`.

**Exit.**

```
pnpm --filter @void/connectors test
node scripts/undo-postgres.mjs
# expected: "replayed 3 compensations; pg_dump diff: 0 lines"
```

**Budget.** About 1.6M tokens.

**Traps.** The human edit between capture and replay. The naive replay overwrites it. The correct behaviour is to detect the drift, refuse, and say what changed. The third skeptic exists to force this.

WP-07: connector, S3

**Goal.** The connector where the class depends on configuration, proved against a real object store: delete marker removal when versioning is on, refusal with a clear reason when it is off.

**Needs.** WP-06 for the interfaces. A local S3 compatible store (MinIO or `s3rver`); if neither runs in this environment, a disposable AWS account with a spend cap that you create. The scout step establishes which.

**Shape.** One workflow, three phases, 9 agents.

| PHASE     | PATTERN                                        | AGENTS |
|-----------|------------------------------------------------|--------|
| Implement | pipeline over 3 modules in worktrees, 1 merger | 4      |
| Verify    | 2 runners, versioning on and off               | 2      |
| Break     | 3 skeptics                                     | 3      |

**Agents.**

- Builders: `s3/classify.ts` (reads the versioning fact, declared or probed), `s3/capture.ts` (for a `PUT` over an existing key, record the prior version id; for a delete, the marker id), `s3/inverse.ts` (delete the marker, or restore the prior version as current).

- Runners: bucket with versioning enabled, delete through the proxy, replay, `HeadObject` succeeds. Bucket with versioning suspended, the same delete is classified R3 and held, and the hold message says why.
- Skeptics: a prefix delete of 500 keys where 3 lack a prior version, a key deleted by version id explicitly, and a bucket policy that denies the probe.

**Produces.** `packages/connectors/s3` .

**Exit.**

```
node scripts/undo-s3.mjs --versioning on      # "object restored, etag matches"
node scripts/undo-s3.mjs --versioning off     # "held: r3, no prior version"
```

**Budget.** About 1.0M tokens.

**Traps.** Batch classification. The worst case in a batch is the class of the batch, and the hold message must name the three keys that made it R3, not just the count.

WP-08: probes and blast radius

**Goal.** Replace declared facts with measurement where a read is available, and put a real number in every hold notification.

**Needs.** WP-06, WP-07.

**Shape.** One workflow, three phases, 12 agents.

| PHASE            | PATTERN                                                    | AGENTS |
|------------------|------------------------------------------------------------|--------|
| Design           | 2 designs of the probe cache and staleness policy, 1 judge | 3      |
| Implement        | pipeline over 4 modules in worktrees, 1 merger             | 5      |
| Verify and break | 1 accuracy runner, 3 skeptics                              | 4      |

**Agents.**

- Designers: cache keyed by target and fact, TTL per fact kind, what to do when a probe fails (fall back to declared, and say so in the ledger).
- Builders: `probe/postgres.ts` (counting `SELECT` in an aborted transaction, cascade counts per table), `probe/s3.ts` ( `ListObjectsV2` on the prefix, versioning per bucket, prior version per key with concurrency limits), `probe/upstream.ts` (ask the upstream MCP server's own read tools for the fact, the option 2 from `BUILD-PLAN.md` section 7.1), `probe/cache.ts` .
- Accuracy runner: for 20 generated statements, compares the probe's row count to the count after actually executing on a copy. Returns the max error. Zero is the bar for Postgres; S3 prefix counts may lag by the listing consistency window and the runner says so.

- **Skeptics:** a probe that takes longer than the hold timeout, a predicate with side effects ( `SELECT` calling a volatile function), and a probe result that is stale because the table changed between probe and call.

**Produces.** `packages/probes` , blast radius in every hold, `void probe` .

**Exit.**

```
node scripts/probe-accuracy.mjs      # "postgres max error 0 rows over 20 statements"
node scripts/moment.mjs              # hold message now carries "41,883 rows across 3 tables"
```

**Budget.** About 1.4M tokens.

**Traps.** A counting `SELECT` of a predicate that calls a function with side effects executes those side effects. The Postgres probe runs as a role with no write grants and inside a transaction that is always aborted, and the second skeptic proves both.

WP-09: the CLI

**Goal.** One binary that runs the proxy, lists holds, verifies the ledger, replays, tests a policy against a transcript, and exports. The CLI is how the product gets debugged for the next year, so it is built before any dashboard.

**Needs.** WP-06. WP-07 and WP-08 improve it but are not required.

**Shape.** One workflow, three phases, 10 agents.

| PHASE     | PATTERN                                                                | AGENTS |
|-----------|------------------------------------------------------------------------|--------|
| Design    | 1 designer for command surface and output conventions                  | 1      |
| Implement | pipeline over 6 commands in worktrees, 1 merger                        | 7      |
| Verify    | 2 runners: a scripted session, and a fresh install on a clean checkout | 2      |

**Agents.**

- **Designer:** `void run` , `void approvals` , `void ledger` , `void replay` , `void policy test` , `void export` . Human output by default, `--json` for everything. Exit codes that mean something.
- **Builders:** one per command, each with tests against the package APIs.
- **Runners:** one walks through a full session using only the CLI; one installs from a clean clone and runs `void --help` and `void run` against the reference server within five minutes, timing it.

**Produces.** `packages/cli` , `npx void` .

**Exit.**

```
pnpm --filter @void/cli test
bash scripts/fresh-install.sh      # "first hold in 4m12s" or better
```

**Budget.** About 1.1M tokens.

## WP-10: control plane, API

**Goal.** The HTTP surface the dashboard and future integrations use: a live feed, approvals, ledger browsing with verification status, replay preview.

**Needs.** WP-09.

**Shape.** One workflow, three phases, 11 agents.

| PHASE     | PATTERN                                                                  | AGENTS |
|-----------|--------------------------------------------------------------------------|--------|
| Design    | 1 designer for the OpenAPI contract, extending <code>lib/api-spec</code> | 1      |
| Implement | pipeline over 5 route groups in worktrees, 1 merger                      | 6      |
| Break     | 4 skeptics                                                               | 4      |

### Agents.

- Designer: extends the existing OpenAPI file; `GET /feed` as SSE, `GET /holds`, `POST /holds/:id/{approve,cancel}`, `GET /ledger`, `GET /ledger/verify`, `POST /replay/preview`, `POST /replay`. Bearer token auth from an environment variable, one organisation.
- Builders: one per route group, reusing the Express patterns already in `artifacts/api-server`.
- Skeptics: approve a hold twice, replay preview for a range that spans a drift, a feed client that disconnects mid event, and a token in the query string instead of the header.

**Produces.** `apps/control-plane/api`, updated OpenAPI, generated client.

### Exit.

```
pnpm --filter @void/control-plane-api test
curl -N localhost:8081/feed | head -3      # three events from a live proxy
```

**Budget.** About 1.3M tokens.

## WP-11: control plane, UI

**Goal.** The smallest dashboard someone who is not you can use: feed, approvals with blast radius and plan, ledger with verification, replay with a preview. Built on the site's own design system so it looks like the same product.

**Needs.** WP-10.



| <b>Shape.</b> One workflow, three phases, 13 agents.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                     |        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|--------|
| PHASE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | PATTERN                                                             | AGENTS |
| Design                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 1 information architecture agent, reading Reader D's map from WP-00 | 1      |
| Implement                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | pipeline over 4 pages in worktrees, 1 merger                        | 5      |
| Verify and break                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 1 browser runner, 3 review lenses                                   | 4      |
| Polish                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 3 agents fixing the review findings, in worktrees                   | 3      |
| <b>Agents.</b> <ul style="list-style-type: none"> <li>Pages: feed, approvals, ledger, replay. Each reuses <code>Section</code> , <code>Reveal</code> , the tone tags and the crop mark figures from <code>artifacts/void</code> .</li> <li>Browser runner: Playwright over every page at 320, 768 and 1440, both themes, no console errors, no horizontal overflow, and an approval performed through the UI that a proxy actually honours.</li> <li>Review lenses: accessibility (keyboard only approval), truthfulness (does the UI ever show a number the API did not send), and design consistency against the site.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                     |        |
| <b>Produces.</b> <code>apps/control-plane/web</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                     |        |
| <b>Exit.</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                     |        |
| <pre>node scripts/verify-ui.mjs      # "0 console errors, 0 overflow, approval honoured"</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                     |        |
| <b>Budget.</b> About 1.5M tokens.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                     |        |
| WP-12: taint graph, two workflows <p><b>Goal.</b> Correlate reads to writes across agents so a compensation knows its real scope, and be honest in the product that the result is a superset of the causal set.</p> <p><b>Needs.</b> WP-10.</p> <p><b>Workflow A, design.</b> Three phases, 8 agents. Three independent designs of the data model (what a read records, what a write records, how context membership is decided), from three biases: exact but heavy, cheap but coarse, and privacy preserving. A judge scores them on the questions in <code>BUILD-PLAN.md</code> phase 8. Two skeptics attack the winner with scenarios: an agent that reads a hundred rows and writes one, and two agents in the same session. Output is a design document you read before workflow B starts.</p> <p><b>Workflow B, implement.</b> Three phases, 12 agents. Builders for <code>taint/record.ts</code> , <code>taint/graph.ts</code> , <code>taint/scope.ts</code> and the API and UI additions, in worktrees, then a merger. One runner replays the three agent scenario from the site's own demo and returns the computed</p> |                                                                     |        |

scope. Three skeptics: a read that returned nothing, a write with arguments that came from the model rather than any read, and a scope query over a million entries.

**Produces.** `packages/taint` , scope in the replay preview.

**Exit.**

```
node scripts/taint-scenario.mjs
# expected: "t2 scope: 6 downstream, 2 of them r3 (stripe.payout.create, sendgrid.mail.send)"
```

**Budget.** About 0.8M plus 1.6M tokens.

**Traps.** Superset drift. If the scope grows to include everything, it is useless. The runner reports scope size against ledger size, and a ratio above a threshold fails the package.

WP-13: attestation and the standalone verifier

**Goal.** A signed period export, and a verifier that installs separately and runs without the product, because a verification tool that only runs inside the product it verifies is not evidence.

**Needs.** WP-10.

**Shape.** One workflow, three phases, 9 agents.

| PHASE     | PATTERN                                        | AGENTS |
|-----------|------------------------------------------------|--------|
| Implement | pipeline over 3 modules in worktrees, 1 merger | 4      |
| Verify    | 1 runner on a clean machine image              | 1      |
| Break     | 4 forgers                                      | 4      |

**Agents.**

- Builders: `export.ts` (period selection, the summary block from the site's attestation page, JSONL out), `frames.ts` (the five frame mappings as data, each field named against the ledger schema), `packages/verify` (a separate package with zero runtime dependencies, published on its own).
- Runner: exports a month, copies only the export and the public key to a clean checkout with nothing else installed, runs `npx void-verify` , returns the result.
- Forgers: same four attacks as WP-03, but against the export file.

**Produces.** `void export` , `packages/verify` , `docs/frames.md` .

**Exit.**

```
void export --period 2026-09 --sign > attest.jsonl
npx void-verify attest.jsonl --key prod.pub # "14203 links intact, signature valid"
```

**Budget.** About 1.0M tokens.

WP-14: SDK wrap

**Goal.** The second deployment shape: `void_.wrap(tools)` returns the same tool objects with preflight, capture and ledger writes attached, for agents that do not use MCP.

**Needs.** WP-13. Only after the MCP proxy has a real user.

**Shape.** One workflow, three phases, 11 agents.

| PHASE      | PATTERN                                                            | AGENTS |
|------------|--------------------------------------------------------------------|--------|
| Understand | 3 readers, one per framework's tool shape                          | 3      |
| Implement  | pipeline over 3 adapters plus the core wrap in worktrees, 1 merger | 5      |
| Verify     | 3 runners, one per framework, each running the moment scenario     | 3      |

**Agents.**

- Readers: how a tool is defined and invoked in the Vercel AI SDK, the OpenAI Agents SDK, and LangGraph. Returns the minimal adapter interface.
- Builders: `wrap.ts` (framework neutral), three adapters.
- Runners: the WP-05 moment, through each framework.

**Produces.** `packages/sdk` .

**Exit.**

```
node scripts/moment.mjs --shape sdk --framework vercel|openai|langgraph
```

**Budget.** About 1.3M tokens.

WP-15: hardening, docs, release

**Goal.** Ship version 0.1 in a state someone can install, trust, and cite.

**Needs.** WP-09 at minimum, WP-13 ideally.

**Shape.** One workflow, four phases, 14 agents.

| PHASE           | PATTERN                                                            | AGENTS  |
|-----------------|--------------------------------------------------------------------|---------|
| Security review | 5 lenses over the whole codebase, structured findings              | 5       |
| Fix             | pipeline over confirmed findings in worktrees, 1 merger            | up to 4 |
| Docs            | 3 writers: quickstart, policy reference, connector authoring guide | 3       |
| Release         | 1 packager, 1 fresh install runner                                 | 2       |

**Agents.**

- Security lenses: credential handling and key material, injection through tool arguments into probes, SSRF from probe URLs, ledger key rotation and revocation, and the approval channel as an attack surface. Each returns findings with a repro; only findings that reproduce get fixed.
- Writers: quickstart that reaches the first hold in five minutes, the policy reference generated from the schema, and a connector guide that a stranger can follow to add a fourth connector.
- Packager: npm publish dry run, a Docker image, GitHub Actions running the full exit criteria on every push, and the MCP registry manifest.
- Fresh install runner: a clean machine, the quickstart, a timer.

**Produces.** `v0.1.0` , published packages, CI, the registry listing manifest.

**Exit.**

```
gh run view --latest           # green
bash scripts/fresh-install.sh  # under five minutes to first hold
npm view @void/cli version     # 0.1.0
```

**Budget.** About 1.8M tokens.

## 5. Totals

| PACKAGE                     | WORKFLOWS | AGENTS      | TOKENS      |
|-----------------------------|-----------|-------------|-------------|
| WP-00 scaffold              | 1         | 9           | 0.7M        |
| WP-01 proxy stdio           | 1         | 14          | 1.5M        |
| WP-02 proxy HTTP            | 1         | 11          | 1.2M        |
| WP-03 ledger                | 1         | 9           | 0.9M        |
| WP-04a registry runtime     | 1         | 10          | 1.1M        |
| WP-04b registry expansion   | 1         | ~500        | 8.0M        |
| WP-05 policy and hold       | 1         | 14          | 1.6M        |
| WP-06 Postgres connector    | 1         | 13          | 1.6M        |
| WP-07 S3 connector          | 1         | 9           | 1.0M        |
| WP-08 probes                | 1         | 12          | 1.4M        |
| WP-09 CLI                   | 1         | 10          | 1.1M        |
| WP-10 control plane API     | 1         | 11          | 1.3M        |
| WP-11 control plane UI      | 1         | 13          | 1.5M        |
| WP-12 taint graph           | 2         | 20          | 2.4M        |
| WP-13 attestation           | 1         | 9           | 1.0M        |
| WP-14 SDK wrap              | 1         | 11          | 1.3M        |
| WP-15 hardening and release | 1         | 14          | 1.8M        |
| <b>Total</b>                | <b>18</b> | <b>~690</b> | <b>~29M</b> |

Without WP-04b, which is optional and batchable, the build is about 21M tokens across 17 workflows and roughly 190 agents.

At the rates in `BUILD-PLAN.md` section 3 for the top tier model, with heavy caching of the context pack, 21M tokens is in the region of \$300 to \$800 of model spend for the whole build. Inside this Claude Code session the cost is carried by the plan rather than metered per token, and the figure is here so the two ways of running it can be compared.

The calendar estimate does not change from `BUILD-PLAN.md` : what the workflows compress is the typing, not the verification. Each package still ends with you reading exit evidence and deciding, and packages from WP-05 onward are verified against real systems that take real time to set up.

## 6. What I need from you, and when

| BEFORE | WHAT                                                                  | WHY                                                  |
|--------|-----------------------------------------------------------------------|------------------------------------------------------|
| WP-00  | The sentence "run WP-00"                                              | Explicit opt in per workflow                         |
| WP-04a | Fifteen minutes to skim the prose to structure migration              | The one merge that needs human eyes                  |
| WP-05  | Optional: a Slack incoming webhook URL for a test channel             | The Slack approval channel; CLI is enough without it |
| WP-07  | If MinIO does not run here: a disposable AWS account with a spend cap | Real S3 verification                                 |
| WP-15  | An npm organisation name, and a GitHub Actions secret for publishing  | Release                                              |
| Any    | A Stripe test mode key, only when a Stripe connector is scheduled     | Not in this plan's first pass                        |

Nothing in this plan needs a production credential of any kind, and nothing should be given one.

## 7. How this plan changes

This document is the contract for the build, and it will be wrong in places. When a package finishes with evidence that changes a later package, the change is made here in the same commit, with a line in `DECISIONS.md` saying what was learned. The plan is edited, not abandoned, and the edit is visible in history.